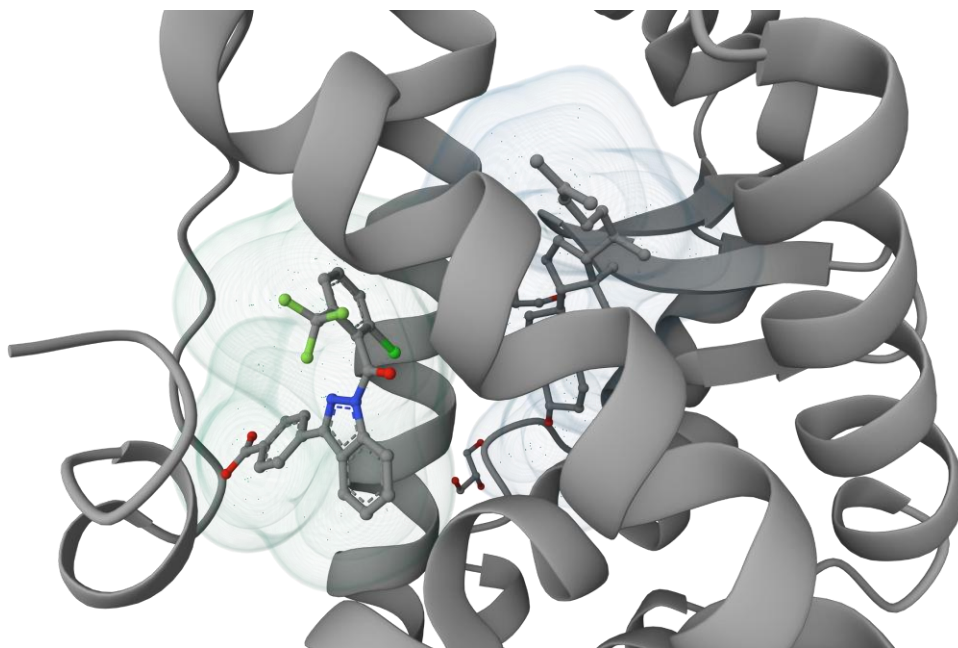


AlloPockets



Francho Nerín-Fonz, Alexios Chatzigoulas & Zoe Cournia

Biomedical Research Foundation

Academy of Athens

URL

Nerín-Fonz F., Chatzigoulas A. and Cournia Z. AlloPockets: a high-performance deep learning model for predicting allosteric sites. Journal 2025; doi

Nerín-Fonz F., Chatzigoulas A. and Cournia Z. A web-based server for predicting allosteric site prediction and pathway computation. Journal 2025; doi

Contents

1. Methodology	3
2. Input.....	4
2.1. Allosteric pocket prediction	4
i. Input structure.....	4
a) PDB ID or UniProt ID	4
b) Protein structure.....	4
ii. Viewer	5
iii. Input chain	5
iv. Job ID.....	5
2.2. Job history	6
a) Job ID.....	6
b) Job history table.....	6
3. Output.....	7
3.1. View results	7
3.2. Download results.....	7
4. Allosteric pathways.....	9
4.1. Pathways calculation.....	9
4.2. Pathways visualization.....	10
5. API.....	12
5.1. gradio_client package.....	12
i. Input	12
ii. Output	12
5.2. cURL	13
i. Input	13
ii. Output	13
6. Bibliography	14

1. METHODOLOGY

AlloPockets implements machine learning to predict allosteric pockets in protein structures, and extends the analysis by tracing potential allosteric communication pathways from these sites to distal regions of the protein. The prediction model was trained on a dataset obtained from curated and corrected allosteric site annotations comprising more than 3000 structures, combined with descriptors covering all levels of protein information – sequence, structure, and dynamics (REF).

When a protein structure is entered into AlloPockets, it is first parsed and adapted to the expected input format, and descriptors are computed with multiple software. MMSeqs¹ and HHBlits² (through the moleculekit package³) are used to obtain sequence-level evolutionary information. Residue type-based scales of values (for the 20 standard amino acids) are extracted from Graphein⁴⁻⁶, and structure-based descriptors are calculated with FreeSASA⁷, DSSP⁸, Biopython⁹, PyRosetta¹⁰, and Melodia¹¹, covering properties such as secondary structure and geometry, exposure, and estimated hydrogen bonding or alanine-mutation energies. ProDy¹² and a previous implementation of transfer entropy calculation¹³ provide dynamical features based on Elastic Network Models. The residue-level descriptors are integrated with the pockets computed by Fpocket¹⁴ and their descriptors, bringing the total feature count to 188. The combined descriptors are passed to the trained neural network classifier at the core of AlloPockets, which outputs probabilities for each candidate pocket being allosteric. Predicted pockets are ranked accordingly, and the results are returned as a table and visualized with the Mol* viewer¹⁵⁻¹⁷ in the web interface built with Gradio¹⁸.

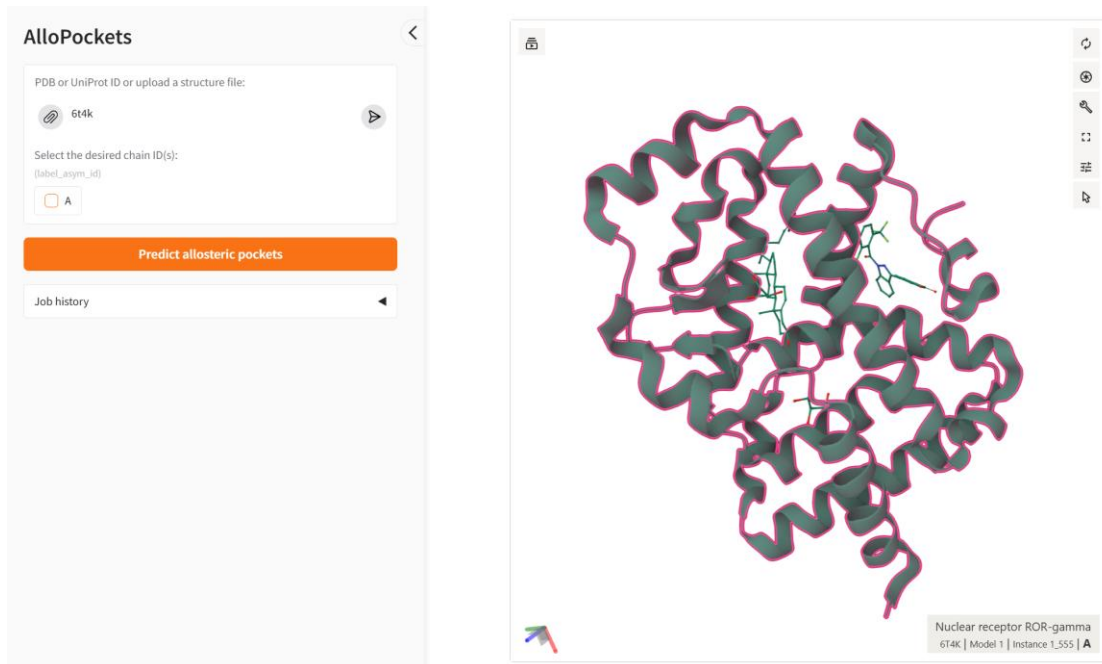
Beyond site prediction, AlloPockets also provides an option to perform allosteric pathway analysis. These are based on allosteric networks built from Elastic Network Models calculations, as implemented by correlationplus and ProDy.

Results are visualized directly in the web server. Predicted allosteric pockets are mapped onto the input structure, and pathway connections are displayed linking the residues. A unique Job ID is assigned to each job, enabling users to retrieve and share results at a later time. Final outputs, including residue annotations, predicted pocket scores, and pathway data, can be downloaded in tabular and graphical formats.

In addition to the web interface, AlloPockets can also be accessed programmatically through its API, allowing automated submission of structures and retrieval of results. This enables integration of allosteric site prediction and pathway analysis into larger computational workflows and pipelines. For more details regarding the methodology please refer to (REF).

2. INPUT

2.1. Allosteric pocket prediction



i. Input structure

a) PDB ID or UniProt ID

The users interested to predict allosteric pockets proceed with the calculation by providing a PDB ID of the protein structure to use, or a UniProt ID to use the structural model of the protein in the AlphaFold Protein Structure Database, and then hit the button to submit or the Enter key.

This close-up shows the input field with the text 'e.g., 6t4k or P51449' and a submit button (arrow icon). The input field is highlighted with a red box.

b) Protein structure

Alternatively, users may also choose to upload their own protein structure by hitting the attachment button, and then the button to submit or the Enter key.

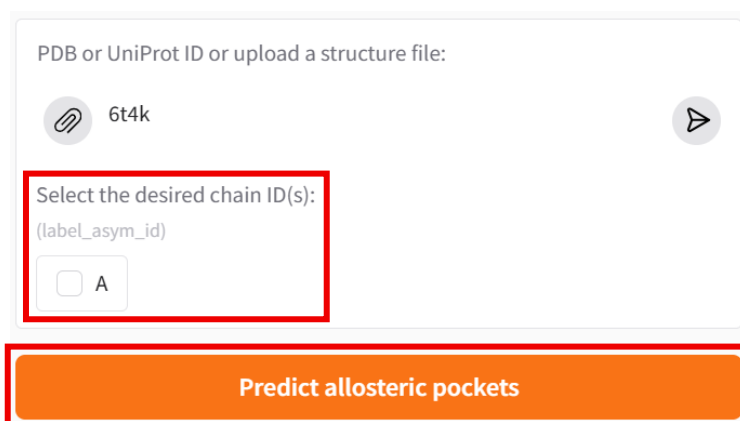
This close-up shows the input field with the text 'e.g., 6t4k or P51449' and a submit button (arrow icon). The attachment button (paperclip icon) is highlighted with a red box.

ii. Viewer

After submitting the desired structure, it will be displayed on the Mol* viewer (REF) in the website, where its elements can be explored and the IDs of the protein chains of interest can be inspected.

iii. Input chain

After submitting a structure, the IDs (label_asym_id) of the protein chains found in it are displayed to select the ones that will be used for allosteric pocket prediction. After selecting the chain(s), hitting the “Predict allosteric pockets” button will submit the calculation.



PDB or UniProt ID or upload a structure file:

 6t4k 

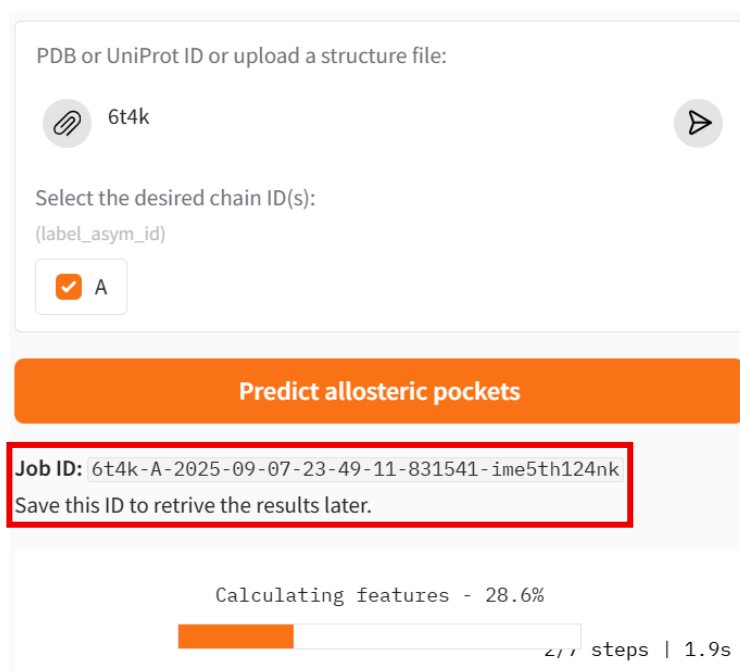
Select the desired chain ID(s):
(label_asym_id)

☐ A

Predict allosteric pockets

iv. Job ID

Immediately after hitting the Predict button, the uniquely assigned Job ID will be displayed so that the user can save it and retrieve the results at a different time. The calculation progress and/or the position in the queue also appears below.



PDB or UniProt ID or upload a structure file:

 6t4k 

Select the desired chain ID(s):
(label_asym_id)

☒ A

Predict allosteric pockets

Job ID: 6t4k-A-2025-09-07-23-49-11-831541-ime5th124nk
Save this ID to retrieve the results later.

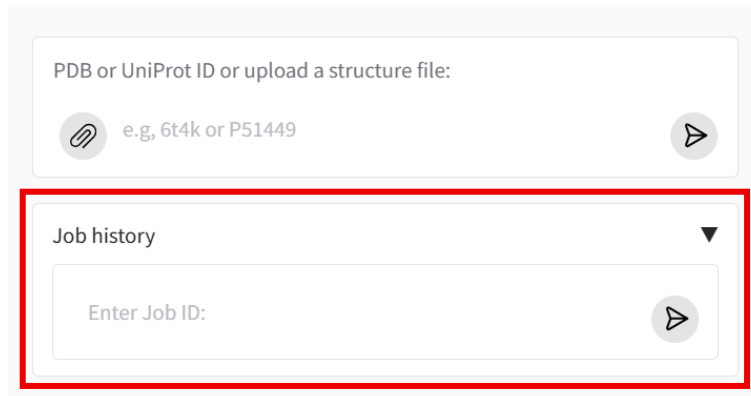
Calculating features - 28.6%

 2/7 steps | 1.9s

2.2. Job history

a) Job ID

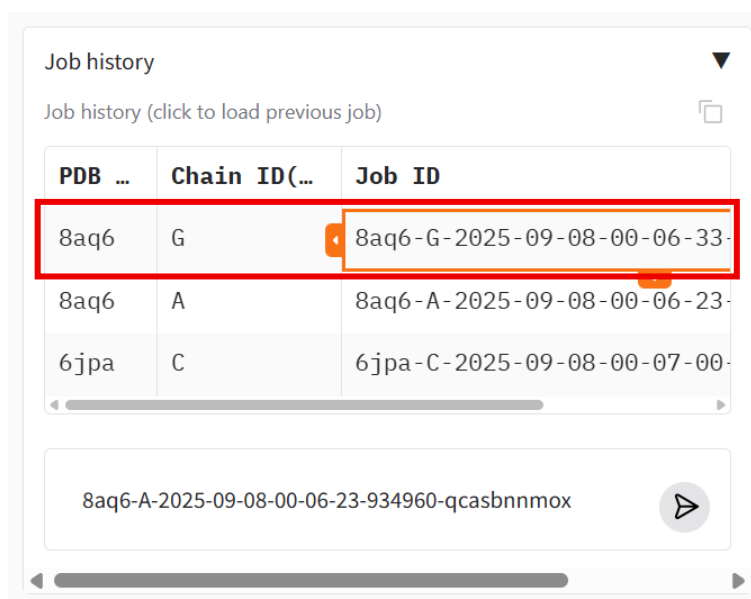
Users can retrieve the results associated to each unique Job ID by entering the code in the box and then hitting the button to submit or the Enter key.



The screenshot shows a web interface with two input fields. The top field is labeled "PDB or UniProt ID or upload a structure file:" and contains a paperclip icon and the text "e.g, 6t4k or P51449". The bottom field is labeled "Job history" and contains the text "Enter Job ID:". Both fields have a submit button with a right-pointing arrow. The "Job history" section is highlighted with a red border.

b) Job history table

Alternatively, when users have populated the Job history table with calculations performed in the same session, previous results can be retrieved by clicking on a row.



The screenshot shows a web interface with a table titled "Job history". The table has three columns: "PDB ...", "Chain ID(...)", and "Job ID". The first row is highlighted with a red border. Below the table is a text input field containing the text "8aq6-A-2025-09-08-00-06-23-934960-qcasbnnmox" and a submit button with a right-pointing arrow.

PDB ...	Chain ID(...)	Job ID
8aq6	G	8aq6-G-2025-09-08-00-06-33
8aq6	A	8aq6-A-2025-09-08-00-06-23
6jpa	C	6jpa-C-2025-09-08-00-07-00

3. OUTPUT

AlloPockets

PDB or UniProt ID or upload a structure file:

6t4k

Select the desired chain ID(s):

(label_asym_id)

☒ A

Predict allosteric pockets

Job ID: 6t4k-A-2025-09-16-12-22-01-638486-kc3dmkv4oqc

Save this ID to retrieve the results later.

Predicted allosteric pockets (click to view)

	Pocket	Allosteric score	Residues
1	pocket:	0.7952967286109924	A: TRP:73, AL
2	pocket:	0.458160459952698	A: TYR:37, CY
3	pocket:	0.003291118890047073	A: GLU:115, P
4	pocket:	0.000695471011567860	A: LEU:45, LE

ALLOSTERIC PATHWAYS

Download results files

Job history

6t4k

pocket2: black

A: TRP:73, ALA:77, LEU:80, THR:81, ILE:84, GLN:85, VAL:88, LEU:109, LYS:110, ALA:113, MET:114, VAL:236, LEU:239, GLN:240, GLN:243, ILE:248, VAL:250, ALA:252, ALA:253, PHE:254, LEU:257, TYR:258, LEU:261, PHE:262

3.1. View results

After the prediction finishes running, a table shows the pockets ranked by the AlloPockets model and the residues that constitute them, while the top pocket is automatically represented in the structure. The table can be copied by hitting the upper right button, and clicking in a row represents the selected pocket.

	Pocket	Allosteric score	Residues
1	pocket:	0.7952967286109924	A: TRP:73, AL
2	pocket:	0.4581604599952698	A: TYR:37, CY
3	pocket:	0.003291118890047073	A: GLU:115, P
4	pocket:	0.000695471011567860	A: LEU:45, LE

3.2. Download results

Hitting the “Download results files” downloads a .zip file with all the contents produced on the AlloPockets run, which follows this structure:

```
extracted_zip/  
  <pdb>_updated.cif.gz  
  <pdb>.cif  
  predictions.csv  
  predictions.log  
  <pdb>/<pdb>_out/  
    <pdb>_out.cif
```

```

    <pdb>_pockets.pqr
    <pdb>_info.txt
    <pdb>_cif_PYMOL.sh
    <pdb>_cif.pml
    <pdb>_cif_VMD.sh
    <pdb>_cif.tcl
    pockets/
        pocket1_atm.cif
        pocket1_vert.pqr
        [...]
features/<pdb>/
    DSSPF.pkl
    TransferEntropyF.pkl
    HHBlitsF.pkl
    PyRosettaF.pkl
    FreeSASAF.pkl
    GrapheinF.pkl
    ProDyF.pkl
    BiopythonF.pkl
    MelodiaF.pkl

```

<pdb>_updated.cif.gz: SIFTS-standardized compressed .cif file downloaded from the Protein Data Bank in Europe (PDBe).

<pdb>.cif: clean structure file with only the selected protein chains, processed by PDBFixer.

predictions.csv: table of ranked pockets, their residue composition and allosteric score.

predictions.log: log file of the AlloPockets calculations.

<pdb>/<pdb>_out: Fpocket output directory and output files. Note that the pocket numbering is given by Fpocket, while the AlloPockets ranking of Fpocket-numbered pockets is detailed in the predictions.csv file. For more information, refer to <https://github.com/Discngine/fpocket>.

<pdb>_out.cif, <pdb>_pockets.pqr: structure files with pocket alpha spheres.

<pdb>_info.txt: summary of pockets and pocket features.

<pdb>_cif_PYMOL.sh, <pdb>_cif.pml, <pdb>_cif_VMD.sh, <pdb>_cif.tcl: Fpocket visualization files for PyMOL and VMD.

<pdb>/<pdb>_out/pockets: directory of individual pocket structures files.

features/<pdb>: directory storing the featurization calculation results used by AlloPockets.

4. ALLOSTERIC PATHWAYS

Allosteric pathways are the internal communication routes within proteins, that conceptually connect distant functional sites. Uncovering allosteric pathways in structures could capture how allosteric perturbations, such as ligand binding or phosphorylation, can transmit their effect far from its point of origin to influence function and activity, and also identify the residues that are critical for mediating such communication.

To model allosteric pathways computationally, first residue–residue allosteric communication networks are computed from Elastic Network Models, and then potential pathways are obtained calculating shortest paths originating from residues in predicted pockets to other regions of interest, using Dijkstra’s algorithm through the NetworkX Python library¹⁹. The available allosteric networks to obtain pathways from can be computed in the AlloPockets web server using the Dynamical Cross-Correlation (DCC) method as implemented in the correlationplus package²⁰, or the Perturbation Response Scanning (PRS) algorithm implemented in ProDy¹². This analysis highlights potential routes of allosteric signal propagation and provides insight into the functional coupling between sites, potentially aiding hypothesis generation for experimental validation or compound design.

The screenshot displays the AlloPockets web interface. On the left, the 'Predict allosteric pockets' section shows the protein ID '6t4k' and a 'Predict allosteric pockets' button. Below this, a table lists the predicted pockets with their scores and associated residues. A red box highlights the 'ALLOSTERIC PATHWAYS' dropdown menu. To the right, a 3D ribbon diagram of the protein structure is shown, with a specific pocket highlighted in black. The pocket is labeled 'pocket2: black' and lists the residues: A: TRP:73, ALA:77, LEU:80, THR:81, ILE:84, GLN:85, VAL:88, LEU:109, LYS:110, ALA:113, MET:114, VAL:236, LEU:239, GLN:240, GLN:243, ILE:248, VAL:250, ALA:252, ALA:253, PHE:254, LEU:257, TYR:258, LEU:261, PHE:262.

	Pocket	Allosteric score	Residues
1	pocket:	0.7952967286189924	A: TRP:73, AL
2	pocket:	0.4581604599952698	A: TYR:37, CY
3	pocket:	0.003291118890047073	A: GLU:115, P
4	pocket:	0.000695471011567860	A: LEU:45, LE

4.1. Pathways calculation

After the successful prediction of allosteric pockets in a structure (or loading of previous results using the Job history or a Job ID), the “ALLOSTERIC PATHWAYS” dropdown allows the user to compute allosteric pathways and visualize them. The user can choose between correlationplus’ Dynamical Cross-Correlation and ProDy’s Perturbation Response Scanning as allosteric network construction methods from which to obtain the allosteric pathways. Then, hitting the “Calculate allosteric pathways” proceeds with the calculations, computing the chosen allosteric network and then obtaining the pathways as

the shortest paths of the network that originate from the selected and visualized pocket, using Dijkstra's algorithm implemented in NetworkX.

ALLOSTERIC PATHWAYS
▼

View the allosteric pathways originating from the predicted pocket towards spatially distant residues in the structure.

The allosteric network can be chosen between [correlationplus](#) (Dynamical Cross-Correlation from Elastic Network Models) or ProDy's [Perturbation Response Scanning](#). Acknowledge them appropriately. The shortest paths between residues of the source pocket and other spatially distant residues (given a distance threshold) are calculated using [NetworkX](#) with Dijkstra's algorithm.

Choose an allosteric network to calculate pathways:

☐ Dynamical Cross-Correlation

☐ Perturbation Response Scanning

Calculate allosteric pathways

4.2. Pathways visualization

Automatically after the calculation finishes, the top 5 pathways (top shortest paths) between residues of the selected pocket and spatially distant residues (at a minimum spatial distance, default 20 Å) are represented in the structure and shown in the Pathways table, which also details for each pathway the colour used in the representation, the length, and the residues that constitute it.

using [NetworkX](#) with Dijkstra's algorithm.

Choose an allosteric network to calculate pathways:

☒ Dynamical Cross-Correlation

☐ Perturbation Response Scanning

Calculate allosteric pathways

Minimum distance (Å) between pocket residues and target residues

20

Number of pathways to visualize

5

Pathways

	Pathway	Color	Path length	Residue
1	P104_toj	blue	0.88989950813766	A: MET:
2	P143_toj	cyan	0.89823086715839	A: MET:
3	P144_toj	green	0.90100837464761	A: MET:
4	P146_toj	yellow	0.90125668826852	A: MET:
5	P182_toj	red	1.05948180099220	A: LYS:

Download results files

Job history

6t4k
pocket2: black
A: TRP:73, ALA:77, LEU:80, THR:81, ILE:84, GLN:85, VAL:88, LEU:109, LYS:110, ALA:113, MET:114, VAL:236, LEU:239, GLN:240, GLN:243, ILE:248, VAL:250, ALA:252, ALA:253, PHE:254, LEU:257, TYR:258, LEU:261, PHE:262

Use via API · Built with Gradio · Settings

The pathways are represented in the protein structure as coloured thick lines connecting the residues that form the pathway. Hovering over the lines or cylinders will show the pathway rank and length on the bottom right of the visualization box, and hovering over the spheres of the residues' alpha-Carbons will show the residue name and number.

The number of top pathways, as well as the minimum distance threshold between residues of the pocket and “target” or “sink” pathway end-point residues, can be adjusted with the sliders. The table showing the pathways and the visualization will be automatically updated when clicking outside of the sliders. Note that, in some cases, the size of the protein could be smaller than the selected distance threshold, even the default 20 Å, which would need to be reduced so that pathways meet the criteria and avoid visualization errors.

Minimum distance (Å) between pocket residues and target residues

10

↺

1

100

Number of pathways to visualize

7

↺

1

21

Pathways

	Pathway	Color	Path length	Residu
1	P98_top	blue	0.3889902500290990	A: MET
2	P18_top	cyan	0.403844653152692	A: CYS
3	P14_top	green	0.4237303159304398	A: TYR
4	P13_top	yello	0.4306397015116471	A: SER
5	P156_top	red	0.4487361386502468	A: GLY
6	P71_top	purpl	0.461020404006919	A: VAL
7	P99_top	grey	0.461819455338987	A: MET

5. API

Users can submit and obtain predictions using the AlloPockets API, through the streamlined `gradio_client` Python (or JavaScript) package, or with classic command-line API querying methods.

5.1. `gradio_client` package

i. Input

To submit a structure for prediction, users can provide one of PDB ID (or UniProt ID) or a path to a local protein structure file (using `gradio_client`'s `handle_file`), in addition to a selection of protein chains to run the calculations on.

```
from gradio_client import Client, handle_file

client = Client(http://127.0.0.1:8998/, download_files="my_path")
jobid, resultsZIP = client.predict(
    pdb={"text": "<PDB or UP ID>", "files": [handle_file("<file>")]},
    chainid="<list of chains>",
    api_name="/predict"
)
```

pdb (dictionary): One of PDB ID (or UniProt ID) or a path to a local protein structure file, in a dictionary structure.

Example PDB ID: `pdb={"text": "6t4k", "files": []}`

Example file: `pdb={"text": "", "files": [handle_file("/path/to/file.pdb")]}`

chainid (string): Comma-separated list of the desired protein chain IDs (`label_asym_id`) of the structure to run predictions on: Example: `chainid="A,B"`

ii. Output

jobid (string): Unique Job ID of the calculations, which can be used to explore results on the website as well.

resultsZIP (string): Local path where the .zip file containing the prediction results has been downloaded into. Default is `/tmp`, and it can be changed by specifying a download path as `download_files="path"` in the `Client()` object initialization. The contents can be extracted to an `output folder` with:

```
import zipfile

with zipfile.ZipFile(resultsZIP, "r") as f:
    f.extractall("<output folder>")
```

5.2. cURL

i. Input

To submit a structure for prediction using the cURL, users can use the call below providing inside “data” one of PDB ID (or UniProt ID) or a path to a local protein structure file, in addition to a selection of protein chains to run the calculations on.

The first and second arguments in “data” correspond to the `pdb` and `chainid` arguments, respectively, in the `gradio_client` call.

```
curl -s -X POST http://127.0.0.1:8998/gradio_api/call/predict \
-H "Content-Type: application/json" \
-d '{
  "data": [
    {"text": "<PDB or UP ID>", "files": [handle_file("<file>")]},
    "<list of chains>"
  ]
}' \
| awk -F'""' '{print $4}' \
| xargs -I{} curl -N http://127.0.0.1:8998/gradio_api/call/predict/{}'
```

ii. Output

The command waits until the results are returned, which take the following form:

```
data: ["8aq6-G-2025-09-15-18-15-48-121954--8051716171144663973",
{"path":
"/tmp/gradio/499b2b5c044dd095bab8c8ef1c64c728978b6d2bfb84feb3c968c8891
3eadab4/8aq6-G-2025-09-15-18-15-48-121954--8051716171144663973.zip",
"url":
"http://127.0.0.1:8998/gradio_api/file=/tmp/gradio/499b2b5c044dd095bab
c88ef1c64c728978b6d2bfb84feb3c968c88913eadab4/8aq6-G-2025-09-15-18-15-
48-121954--8051716171144663973.zip", "size": null, "orig_name": "8aq6-
G-2025-09-15-18-15-48-121954--8051716171144663973.zip", "mime_type":
null, "is_stream": false, "meta": {"_type": "gradio.FileData"}}]
```

Highlighted in bold are the Job ID, and the URL to download the .zip file containing all results, respectively.

6. BIBLIOGRAPHY

- (1) Steinegger, M.; Söding, J. MMseqs2 Enables Sensitive Protein Sequence Searching for the Analysis of Massive Data Sets. *Nat. Biotechnol.* **2017**, 35 (11), 1026–1028. <https://doi.org/10.1038/nbt.3988>.
- (2) Remmert, M.; Biegert, A.; Hauser, A.; Söding, J. HHblits: Lightning-Fast Iterative Protein Sequence Searching by HMM-HMM Alignment. *Nat. Methods* **2012**, 9 (2), 173–175. <https://doi.org/10.1038/nmeth.1818>.
- (3) Doerr, S.; Harvey, M. J.; Noé, F.; De Fabritiis, G. HTMD: High-Throughput Molecular Dynamics for Molecular Discovery. *J. Chem. Theory Comput.* **2016**, 12 (4), 1845–1852. <https://doi.org/10.1021/acs.jctc.6b00049>.
- (4) Jamasb, A. R.; Torné, R. V.; Ma, E. J.; Du, Y.; Harris, C.; Huang, K.; Hall, D.; Lio, P.; Blundell, T. L. Graphein - a Python Library for Geometric Deep Learning and Network Analysis on Biomolecular Structures and Interaction Networks; 2022.
- (5) Meiler, J.; Müller, M.; Zeidler, A.; Schmäschke, F. Generation and Evaluation of Dimension-Reduced Amino Acid Parameter Representations by Artificial Neural Networks. *Mol. Model. Annu.* **2001**, 7 (9), 360–369. <https://doi.org/10.1007/s008940100038>.
- (6) Gasteiger, E.; Hoogland, C.; Gattiker, A.; Duvaud, S.; Wilkins, M. R.; Appel, R. D.; Bairoch, A. Protein Identification and Analysis Tools on the ExPASy Server. In *The Proteomics Protocols Handbook*; Walker, J. M., Ed.; Humana Press: Totowa, NJ, 2005; pp 571–607. <https://doi.org/10.1385/1-59259-890-0:571>.
- (7) Mitternacht, S. FreeSASA: An Open Source C Library for Solvent Accessible Surface Area Calculations. *F1000Research* February 18, 2016. <https://doi.org/10.12688/f1000research.7931.1>.
- (8) Kabsch, W.; Sander, C. Dictionary of Protein Secondary Structure: Pattern Recognition of Hydrogen-Bonded and Geometrical Features. *Biopolymers* **1983**, 22 (12), 2577–2637. <https://doi.org/10.1002/bip.360221211>.
- (9) Cock, P. J. A.; Antao, T.; Chang, J. T.; Chapman, B. A.; Cox, C. J.; Dalke, A.; Friedberg, I.; Hamelryck, T.; Kauff, F.; Wilczynski, B.; de Hoon, M. J. L. Biopython: Freely Available Python Tools for Computational Molecular Biology and Bioinformatics. *Bioinformatics* **2009**, 25 (11), 1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>.
- (10) Chaudhury, S.; Lyskov, S.; Gray, J. J. PyRosetta: A Script-Based Interface for Implementing Molecular Modeling Algorithms Using Rosetta. *Bioinformatics* **2010**, 26 (5), 689–691. <https://doi.org/10.1093/bioinformatics/btq007>.
- (11) Montalvão, R. W.; Pitt, W. R.; Pinheiro, V. B.; Blundell, T. L. Melodia: A Python Library for Protein Structure Analysis. *Bioinformatics* **2024**, 40 (7), btae468. <https://doi.org/10.1093/bioinformatics/btae468>.
- (12) Zhang, S.; Krieger, J. M.; Zhang, Y.; Kaya, C.; Kaynak, B.; Mikulska-Ruminska, K.; Doruker, P.; Li, H.; Bahar, I. ProDy 2.0: Increased Scale and Scope after 10 Years of Protein Dynamics Modelling with Python. *Bioinformatics* **2021**, 37 (20), 3657–3659. <https://doi.org/10.1093/bioinformatics/btab187>.
- (13) Hu, F.; Chang, F.; Tao, L.; Sun, X.; Liu, L.; Zhao, Y.; Han, Z.; Li, C. Prediction of Protein Allosteric Sites with Transfer Entropy and Spatial Neighbor-Based Evolutionary Information Learned by an Ensemble Model. *J. Chem. Inf. Model.* **2024**, 64 (15), 6197–6204. <https://doi.org/10.1021/acs.jcim.4c00544>.
- (14) Le Guilloux, V.; Schmidtke, P.; Tuffery, P. Fpocket: An Open Source Platform for Ligand Pocket Detection. *BMC Bioinformatics* **2009**, 10 (1), 168. <https://doi.org/10.1186/1471-2105-10-168>.

- (15) Sehnal, D.; Bittrich, S.; Deshpande, M.; Svobodová, R.; Berka, K.; Bazgier, V.; Velankar, S.; Burley, S. K.; Koča, J.; Rose, A. S. Mol* Viewer: Modern Web App for 3D Visualization and Analysis of Large Biomolecular Structures. *Nucleic Acids Res.* **2021**, 49 (W1), W431–W437. <https://doi.org/10.1093/nar/gkab314>.
- (16) Bittrich, S.; Midlik, A.; Varadi, M.; Velankar, S.; Burley, S. K.; Young, J. Y.; Sehnal, D.; Vallat, B. Describing and Sharing Molecular Visualizations Using the MolViewSpec Toolkit. *Curr. Protoc.* **2024**, 4 (7), e1099. <https://doi.org/10.1002/cpz1.1099>.
- (17) Midlik, A.; Bittrich, S.; Fleming, J. R.; Nair, S.; Velankar, S.; Burley, S. K.; Young, J. Y.; Vallat, B.; Sehnal, D. MolViewSpec: A Mol* Extension for Describing and Sharing Molecular Visualizations. *Nucleic Acids Res.* **2025**, 53 (W1), W408–W414. <https://doi.org/10.1093/nar/gkaf370>.
- (18) Abid, A.; Abdalla, A.; Abid, A.; Khan, D.; Alfozan, A.; Zou, J. Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. arXiv 2019. <https://doi.org/10.48550/ARXIV.1906.02569>.
- (19) Hagberg, A. A.; Schult, D. A.; Swart, P. J. Exploring Network Structure, Dynamics, and Function Using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*; 2008; p 5.
- (20) Tekpinar, M.; Neron, B.; Delarue, M. Extracting Dynamical Correlations and Identifying Key Residues for Allosteric Communication in Proteins by Correlationplus. *J. Chem. Inf. Model.* **2021**, 61 (10), 4832–4838. <https://doi.org/10.1021/acs.jcim.1c00742>.